Cryptographie avec les courbes elliptiques

Comment crypter un échange écrit à l'aide des courbes elliptiques ?

LERINCKX Maxime n°34343

Sommaire

1. Courbe elliptique, groupe commutatif

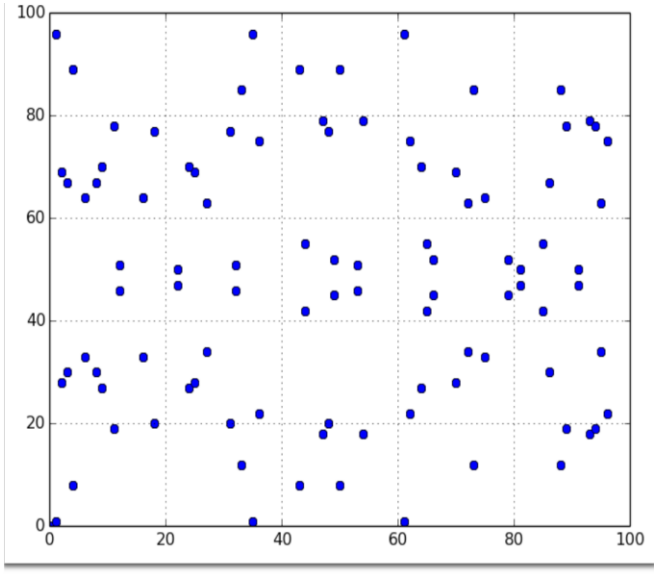
2. Différents algorithmes de cryptages

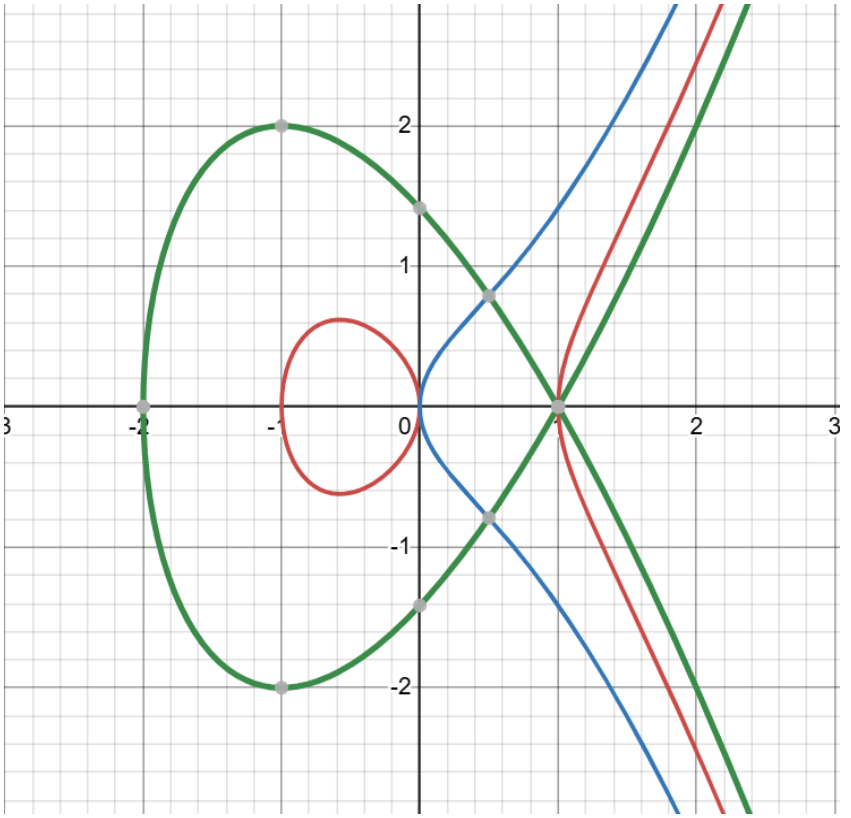
Les courbes elliptiques

1 $y^2 = x^3 - x$

2 $y^2 = x^3 + x$

3 $y^2 = x^3 - 3x + 2$





Les courbes elliptiques

Soit p un nombre premier, et $\mathbb{F}_p$ le corps fini à p éléments.

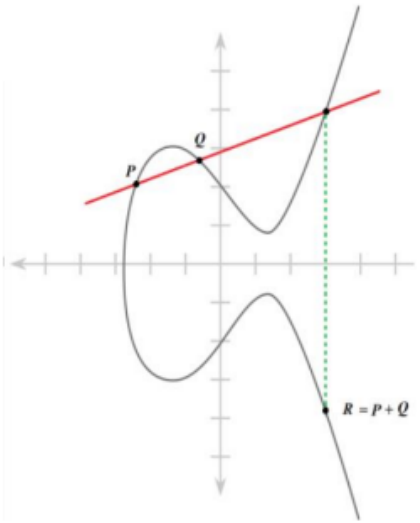
Soient $a, b \in \mathbb{F}_p$ tels que $4a^3 + 27b^2 \not\equiv 0 \pmod{p}$.

$$E(\mathbb{F}_p) = \left\{(x, y) \in \mathbb{F}_p^2 \mid y^2 \equiv x^3 + ax + b \pmod{p}\right\} \cup \{\mathcal{O}\}$$

```
116
117  class CourbeElliptique:
122      def __init__(self, a, b, mod=mod_zero):
137          self.a = a
138          self.b = b
139          self.mod = mod
140
141          self.delta = (-16) * (4 * (a**3) + 27 * (b**2)) % self.mod
142          if not self.delta:
143              raise ValueError("Le delta est nul")
144
```

Loi plus

Cas 1 : $P \neq Q$

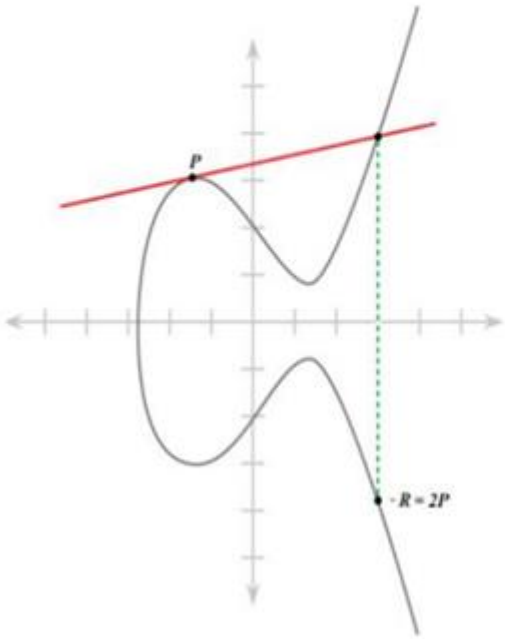


$$\begin{cases} \lambda \equiv \dfrac{y_Q - y_P}{x_Q - x_P} \pmod{m}, \\ x_3 \equiv \lambda^2 - x_P - x_Q \pmod{m}, \\ y_3 \equiv \lambda (x_P - x_3) - y_P \pmod{m}, \\ P + Q = (x_3, y_3) \end{cases}$$

```
284         if P != Q:
285             a_plus = (Q.y - P.y) % self.mod
286             b_plus = (Q.x - P.x) % self.mod
287             l = (a_plus*(inv_mod(b_plus, self.mod))) % self.mod
```

Loi plus

Cas 2 : $P = Q$ (doublement)

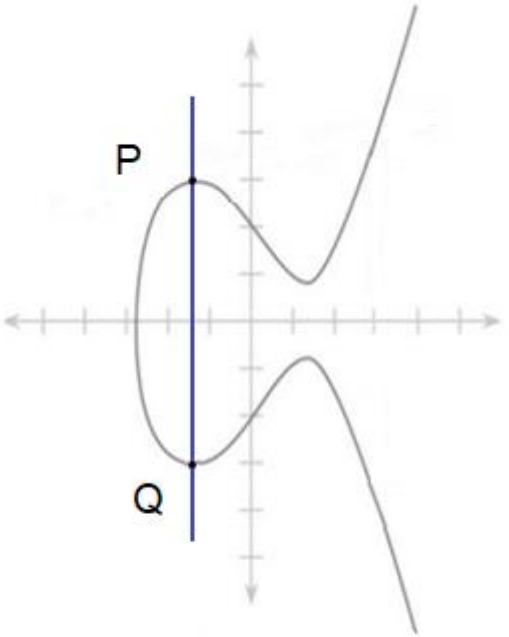


$$\begin{cases} \lambda \equiv \dfrac{3x_P^2 + a}{2y_P} \pmod{m}, \\ x_3 \equiv \lambda^2 - 2x_P \pmod{m}, \\ y_3 \equiv \lambda(x_P - x_3) - y_P \pmod{m}, \\ 2P = (x_3, y_3) \end{cases}$$

```
289         else:  # Si P = Q
290             a_plus = (3*(P.x**2) + self.a) % self.mod
291             b_plus = (2*P.y) % self.mod
292             l = (a_plus*inv_mod(b_plus, self.mod)) % self.mod
```

Loi plus

- Cas 3 P = -Q



$$P + Q = 0$$

```
281         if P == Q.opose():
282             return Point(None, None)
```

Recherche de point

```
180     def __call__(self, x):
194         y2 = (x**3 + self.a * x + self.b) % self.mod
195         try:
196             y_pos = residu_quadratique(y2, self.mod)
197             return y_pos
198 
199         except Exception:
200             raise ValueError(f"Aucune solution pour x = {x} dans Z/{self.mod}Z")
```

```
45  def residu_quadratique(n, p):
46      """
47      Trouve une racine carrée de n modulo p,
48      où p est un premier impair et n est un résidu quadratique modulo p.
49  
50      Arguments:
51         n : int
52             Le nombre pour lequel on cherche la racine carrée modulo p.
53         p : int
54             Le modulo, un nombre premier impair.
55  
56      Retourne:
57         int : La racine carrée de n modulo p.
58  
59      Lève une exception ValueError si n n'est pas un résidu quadratique modulo p.
60  
61      Si p ≡ 3 (mod 4), utilise une méthode simplifiée. Sinon, utilise l'algorithme de Tonelli-Shanks.
62      """
```

Differents algorithmes de cryptages

- Diffie-Hellman + AES (Symetrique)
- ElGamal (Asymetrique)

Diffie-Hellman + AES

- 1. Choix paramètres communs (public)

  Le groupe CE et un générateur G de la courbe elliptique

```
16  class DH_AES:
21      def   init  (self, a, b, mod, G = False):
34          self.CE = ECC.CourbeElliptique(a, b, mod)
35          if not G :
36              self.G = G
37          else:
38              self.G = self.CE.point_aleatoire()
39          assert self.CE.appartient(self.G)
40
```

Diffie-Hellman + AES

- 2. Générations des clés

clés secrètes S_k → clés publiques $P_k = s_k * G$

```
41      def generation_cles(self):
```

```
53          skA = random.randint(1, self.CE.mod)
54          pkA = self.CE.scalaire(skA, self.G)
55          skB = random.randint(1, self.CE.mod)
56          pkB = self.CE.scalaire(skB, self.G)
57          return pkA, skA, pkB, skB
```

Diffie-Hellman + AES

- 3. Etablissement de la clé partagée

$$K = s_k * P_k$$

- 4. Hacher K avec SHA-256 pour obtenir K_{sym}

```
59      def derive_cle_secrete(self, Pk, Sk):
74          K = self.CE.scalaire(Sk, Pk)
75          K_bytes = str(K.x).encode()
76          return hashlib.sha256(K_bytes).digest()
```

Diffie-Hellman + AES

- 5. Chiffrement du message avec AES

  On choisit IV aléatoirement

  Chiffrer avec AES en utilisant K_{sym} et IV

-

- 6. Déchiffrement

  On récupère IV et le texte chiffré et on le déchiffre à l'aide de K_{sym}

```
78      def chiffrement_message(self, message, ksym):
94          ksym = ksym[:32]
95          cipher = AES.new(ksym, AES.MODE_CBC)
96          message_padded = pad(message.encode(), AES.block_size)
97          ciphertext = cipher.encrypt(message_padded)
98          return base64.b64encode(ciphertext).decode(), base64.b64encode(cipher.iv).decode()
```

Test DH_AES

```
Bob envoi à Alice : (492106195314348876782600645074672048435588138409017294203029262540927332648 74,
381951127331532112218682647175810628136986836882941182595382681534656808951 25)
Alice envoi à Bob : (115436927414092072799077419225593494586277331543263199113319256899764636452 82,
346297661205841812617730154603719205503000484898731531727024927553012750947 71)
Alice calcul Ksym : b':
\x16a\x08\xe0\xa3\xc0\x7f\xfdF\x17P\xf4m\xe0\x99\x1e\xee6\x04\x991\xf9X\xad\xa1\x80\xc3\xdb\x06\xd4\xc5'
Bob calcul Ksym : b':
\x16a\x08\xe0\xa3\xc0\x7f\xfdF\x17P\xf4m\xe0\x99\x1e\xee6\x04\x991\xf9X\xad\xa1\x80\xc3\xdb\x06\xd4\xc5'
L'obtention de Ksym a réussi
Alice envoie le message chiffré à Bob
Message chiffré : C1/LItOX1DCWFUthad/HJzEpJcQQ5YNIKLB6kNQhxwQDH4NoYLEDzRaZvq4FkQqtdATDzp1/BufZXp8rYDRGeg==
Bob décrypte le message
Message déchiffré : Je suis heureux que cela fonctionne, vive mon TIPE !
Le chiffrement et déchiffrement ont réussi !
```

Echange de clés

- Protocol de Diffie-Hellman

$$S_{k,A} = \text{random}([\![1, \text{mod}]\!])$$
$$S_{k,B} = \text{random}([\![1, \text{mod}]\!])$$

$$P_{k,A} = S_{k,A} \cdot G$$
$$P_{k,B} = S_{k,B} \cdot G$$

$$\text{Alice} : \ K = S_{k,A} \cdot P_{k,B}$$
$$\text{Bob} : \ K' = S_{k,B} \cdot P_{k,A}$$

$$K = S_{k,A} \cdot (S_{k,B} \cdot G) = S_{k,B} \cdot (S_{k,A} \cdot G) = K'$$

Problème du logarithme discret

- $P = G^k$
- $\Rightarrow k = \log_G(P)$

- Baby-step Giant-step (Daniel Shanks)
  - Complexité en $O(\sqrt{N} * \log(N))$ etapes
  - Pour $N = 2^{256}$ (standard) $\sqrt{N} = 2^{128} \simeq 10^{38}$ opérations
    - 10^{18} opérations/seconde => 10^{15} années

ElGamal

- 1. Choix paramètres communs (public)
  - Un groupe CE et un generateur G de la courbe elliptique

- 2. Génération des clés
  - Clés secrètes $S_k \rightarrow$ Clés publiques $P_k = S_k * G$

ElGamal

- 3. Conversion du message

On convertit le message en une valeur mathématique exploitible notée M

```
161     def message_to_point(self, message):
178         x_original = int.from_bytes(message.encode(), 'big') % self.CE.mod
179         x = x_original
180         decalage = 0
181         boucle_max = 1000
182         boucle = 0
183 
184         while boucle < boucle_max:
185             try:
186                 y = self.CE(x)
187                 return ECC.Point(x, y, self.CE.mod), decalage
188             except ValueError:
189                 x = (x + 1) % self.CE.mod
190                 decalage += 1
191                 boucle += 1
192 
193         raise ValueError("Le message ne peut être converti")
```

ElGamal

- 4. Chiffrement du message

$$M_{chiffré} = M * P_{k,destinataire}^{S_{k,émetteur}}$$

```
216     def chiffrement_message(self, pkA, skB, message):
234         M, decalage = self.message_to_point(message)
235         skB_pkA = self.CE.scalaire(skB, pkA)
236         M_chiffre = self.CE.plus(skB_pkA, M)
237         return M chiffre, decalage
```

- 5. Déchiffrement du message chiffré

$$M_{dechiffré} = M_{chiffré} * (P_{k,émetteur}^{S_{k,déstinataire}})^{-1}$$

```
239     def dechiffrement_message(self, skA, pkB, M_chiffre, decalage=0):
257         skA_pkB = self.CE.scalaire(skA, pkB)
258         skA_pkB_inv = skA_pkB.opose()
259         M_dechiffre = self.CE.plus(M_chiffre, skA_pkB_inv)
260         return self.point_to_message(M_dechiffre, decalage)
```

ElGamal

- 6. Récupération du message

  On convertit la valeur mathématique M en message

```
195     def point_to_message(self, point, decalage=0):
209         x_original = (point.x - decalage) % self.CE.mod
210         try:
211             message = x_original.to_bytes((x_original.bit_length() + 7) // 8,
212             return message
213         except UnicodeDecodeError:
214             return f"[Message illisible], {x_original}"
```

Test EC_ElGamal

```
Alice envoie sa clé publique à Bob :
(4892447859618026872178464816204899823758630916870924726829377650237306220841 1,
6393625960560817543610020835698681024236240311050352834398989463702371258263 4)
Bob chiffre le message grâce à sa clé privée et à la clé publique d'Alice
Bob envoie à Alice sa clé publique :
(4150013642632749948570423471237814520733672896905561751062916516465250266025 3,
4056237315856985735305049825630910393219676565832887824297761775681987031428 2) ; et le message chiffré
: (6197939362605843277863044650829932977960836163394373501862795976513139424064 8,
9305183666156328989978167311519264254292999866418677586398816240825109346715 9) avec le décalage : 0
Alice déchiffre le message en utilisant sa clé privée, la clé publique de Bob, le message chiffré et
le décalage
Out[1]: 'Je suis heureux.'
```

Merci de votre attention

1 Algorithme ElGamal

$y^2 \equiv x^3 + ax + b \pmod{\text{mod}}$
G Point générateur
$S_{k,A} = \text{random}(\llbracket 1, \text{mod} \rrbracket)$
$S_{k,B} = \text{random}(\llbracket 1, \text{mod} \rrbracket)$

$P_{k,A} = S_{k,A} \cdot G$
$P_{k,B} = S_{k,B} \cdot G$
$S_{k,B}P_{k,A} = S_{k,B} \cdot P_{k,A}$
$M_{chiffre} = S_{k,B}P_{k,A} + M$
M message transformé en un point

Alice reçoit $(M_{chiffre}, P_{k,B})$

$S_{k,A}P_{k,B} = S_{k,A} \cdot P_{k,B}$
$M_{dechiffre} = M_{chiffre} - S_{k,A}P_{k,B}$

On retrouve que $M = M_{dechiffre}$ car :
$M_{dechiffre} = M_{chiffre} - S_{k,A}P_{k,B}$
$= S_{k,B}P_{k,A} + M - S_{k,A}P_{k,B}$
$= S_{k,B} \cdot P_{k,A} + M - S_{k,A} \cdot P_{k,B}$
$= M + S_{k,B} \cdot (S_{k,A} \cdot G) - S_{k,A} \cdot (S_{k,B} \cdot G)$
$= M + S_{k,A} \cdot S_{k,B} \cdot G - S_{k,A} \cdot S_{k,B} \cdot G$
$= M$

2 Échange de clés Diffie-Hellman

$S_{k,A} = \text{random}(\llbracket 1, \text{mod} \rrbracket)$
$S_{k,B} = \text{random}(\llbracket 1, \text{mod} \rrbracket)$

$P_{k,A} = S_{k,A} \cdot G$
$P_{k,B} = S_{k,B} \cdot G$

Alice : $K = S_{k,A} \cdot P_{k,B}$
Bob : $K' = S_{k,B} \cdot P_{k,A}$

$K = S_{k,A} \cdot (S_{k,B} \cdot G) = S_{k,B} \cdot (S_{k,A} \cdot G) = K'$

3 Logarithme discret sur courbe elliptique

Soit p un nombre premier, et $\mathbb{F}_p$ le corps fini à p éléments.

Soient $a, b \in \mathbb{F}_p$ tels que $4a^3 + 27b^2 \not\equiv 0 \pmod{p}$.

$E(\mathbb{F}_p) = \left\{(x, y) \in \mathbb{F}_p^2 \mid y^2 \equiv x^3 + ax + b \pmod{p}\right\} \cup \{\mathcal{O}\}$

Soit $G \in E(\mathbb{F}_p)$ un point générateur d'un monogène fini de $E(\mathbb{F}_p)$ d'ordre n

$\langle G \rangle = \{k \cdot G \mid k \in [\![0, n-1]\!]\}$

Donc $\forall P \in \langle G \rangle, \exists! \, k \in [\![0, n-1]\!]$ tel que $P = k \cdot G$

k est appelé logarithme de P en base G et est noté $\log_G(P) = k$

Le problème du logarithme discret sur les courbes elliptiques est :

Trouver $k \in [\![0, n-1]\!]$ tel que $P = k \cdot G$ connaissant G et P

i.e on cherche $log_G(P)$

Il existe deux grand algorithmes conçues pour résoudre le problème du logarithme discret :

3.1 Baby-step Giant-step (Daniel Shanks)

L'algorithme résout le DLP $g^x = h$ en $\mathcal{O}(\sqrt{N} \cdot \log N)$ étapes

Dans notre cas : G d'ordre $N, P \in \langle G \rangle, \exists! k < N$ tel que $P = k \cdot G$

① $n = \lfloor \sqrt{N} \rfloor + 1$

② Baby-step : $[i \cdot G \mid i \in [\![0, n]\!]]$
Giant-step : $[P + (j \cdot n) \cdot (-G) \mid j \in [\![0, n]\!]]$

③ Trouver (i, j) tel que $i \cdot G = P + (n \cdot j) \cdot (-G)$

④ $k = i + n \cdot j$ solution du DLP

Correction :

$P \in \langle G \rangle \Rightarrow \exists k \in [\![0, N-1]\!]$ tel que $P = k \cdot G$

Par Division Euclidiene de k par n : $\exists q \in [\![0, N-1]\!]$ et $r \in [\![0, n-1]\!]$ tel que $K = n \times q + r$ donc :

① comme $K < N$ et $N > \sqrt{N}$

on a : $q = \dfrac{k-r}{n} < \dfrac{N}{n} < n$ car $n - 1 \leq \sqrt{N} < n \implies N < n^2$

d'où $r \cdot G = P + (n \times q) \cdot (-G)$ avec $0 \leq r < n$ et $0 \leq q < n$

Complexité :

Le calcule des deux listes dans ② requiert $3n$ opération de groupe

On trie les deux listes pour trouver un élément commun, ce qui se fait en $\mathcal{O}(n \cdot \log n)$ étapes, or $n \approx \sqrt{N}$

d'où la compléxité totale est en $\mathcal{O}(\sqrt{N} \cdot \log N)$

3.1.1 Remarque

Même si l'algorithme Baby-step Giant-step est théoriquement plus rapide que les attaques naïves (*complexité* $\mathcal{O}(\sqrt{N}\log N)$), il reste impraticable pour les groupes utilisés en cryptographie moderne :

- Pour un ordre $N \approx 2^{256}$ (sécurité standard), $\sqrt{N} = 2^{128}$ opérations sont nécessaires.

- Cela représente environ 10^{38} opérations **et** autant d'espace mémoire (environ 10^{40} octets) pour stocker les résultats.

3.2 Algorithme de Pohlig-Hellman Avec les mêmes notations

① On décompose N en facteur premier : $N = \prod_{i=1}^{r} p_i^{e_i}$

② Pour chaque $p_i^{e_i}$:

Calcule k_i tel que $P_i = k_i \cdot G_i$ avec : $P_i = \frac{N}{p_i^{e_i}} \cdot P$ et $G_i = \frac{N}{p_i^{e_i}} \cdot G$

Utiliser une méthode approprié comme Baby-step Giant-step pour determiner k_i

③ Combinaison des resultats : utiliser le théorème des restes chinois pour reconstuire k à partir des k_i :
On cherche une solution du système $S = \{K \equiv k_i \pmod{p_i^{e_i}}\}_{i \in [\![1,r]\!]}$

```python
#Fichier ECC.py
# -*- coding: utf-8 -*-
"""
Created on Tue Jun 18 16:05:57 2024

@author: maxime
"""

import matplotlib.pyplot as plt
import numpy as np
import random

mod_zero = float('inf')

"""Arithmétique modulaire"""

def inv_mod(n, p):
    """
    Calcule l'inverse modulaire de x modulo 'p' en utilisant l'algorithme d'Euclide étendu.

    Arguments:
        x : int
            Le nombre dont on souhaite trouver l'inverse modulo p.
        p : int
            Le modulo pour lequel on cherche l'inverse de x.

    Retourne:
        int : L'inverse modulaire de x modulo p.

    Lève une exception ValueError si l'inverse modulaire n'existe pas (si le pgcd(x, p) ≠ 1).
    """

    a, r, u, v = n, p, 1, 0

    while r > 0:
        q = a // r
        a, r = r, a % r
        u, v = v, u - q * v

    assert a == 1, "L'inverse modulaire n'existe pas car pgcd(x, p) ≠ 1"

    return u % p

```

```python
def residu_quadratique(n, p):
    """
    Trouve une racine carrée de n modulo p, où p est un premier impair et n est un résidu quadratique modulo p.

    Arguments:
       n : int
           Le nombre pour lequel on cherche la racine carrée modulo p.
       p : int
           Le modulo, un nombre premier impair.

    Retourne:
       int : La racine carrée de n modulo p.

    Lève une exception ValueError si n n'est pas un résidu quadratique modulo p.

    Si p ≡ 3 (mod 4), utilise une méthode simplifiée. Sinon, utilise l'algorithme de Tonelli-Shanks.
    """

    # Vérifie si n est un résidu quadratique modulo p
    if pow(int(n), (p - 1) // 2, p) != 1:  # Si n n'est pas un résidu quadratique
        raise ValueError(f"{n} n'est pas un résidu quadratique modulo {p}")

    # Cas trivial
    if n == 0:
        return 0
    if n == 1:
        return 1

    # Si p ≡ 3 (mod 4), on peut utiliser la formule simplifiée
    if p % 4 == 3:
        return pow(int(n), (p + 1) // 4, p)
```

```
    # Sinon, utiliser l'algorithme de Tonelli-Shanks
    # Recherche de q et s dans p-1 = q * 2^s, où q est impair
    s = 0
    q = p - 1
    while q % 2 == 0:
        s += 1
        q //= 2

    # Initialisation de z
    z = 2
    while pow(z, (p - 1) // 2, p) == 1:
        z += 1

    # Initialisation de x, y, t, m
    x = pow(n, (q + 1) // 2, p)
    y = pow(n, q, p)
    t = pow(n, q, p)
    m = s

    while t != 0 and t != 1:
        # Recherche d'un k
        t2i = t
        i = 0
        for i in range(1, m):
            t2i = pow(t2i, 2, p)
            if t2i == 1:
                break
        # Mise à jour de x, y, t, m
        if i == 0:
            return x
        b = pow(y, 2 ** (m - i - 1), p)
        m = i
        x = (x * b) % p
        y = (y * b * b) % p
        t = (t * b * b) % p

    return x  # Si t == 1, on a trouvé la racine carrée.
```

```python
class CourbeElliptique:
    """
    Représente une courbe elliptique de l'équation y^2 = x^3 + ax + b (mod p).
    """

    def __init__(self, a, b, mod=mod_zero):
        """
        Initialise une courbe elliptique avec les paramètres a, b et mod.

        Arguments:
            a : int
                Le coefficient de x dans l'équation de la courbe elliptique.
            b : int
                Le coefficient constant dans l'équation de la courbe elliptique.
            mod : int, optionnel
                Le modulo de la courbe (par défaut, "mod_zero" est utilisé).

        Lève une exception ValueError si le delta de la courbe est nul.
        """

        self.a = a
        self.b = b
        self.mod = mod

        self.delta = (-16) * (4 * (a**3) + 27 * (b**2)) % self.mod
        if not self.delta:
            raise ValueError("Le delta est nul")

    def __str__(self):
        """
       Retourne une représentation sous forme de chaîne de la courbe elliptique.

       Retourne:
           str : Une chaîne représentant l'équation de la courbe elliptique.
       """
       return f"y^2 = x^3 + {self.a} * x + {self.b}"
```

```
153     def __repr__(self):
154         """
155        Retourne une représentation plus détaillée de la courbe elliptique.
156
157        Retourne:
158            str : Une chaîne détaillant la courbe elliptique et son modulo.
159        """
160
161         return str(self) + " mod " + str(self.mod)
162
163
164     def __eq__(self, autre):
165         """
166         Vérifie si deux courbes elliptiques sont égales.
167
168         Arguments:
169             autre : CourbeElliptique
170                 Une autre courbe elliptique à comparer.
171
172         Retourne:
173             bool : True si les courbes sont égales, False sinon.
174         """
175
176         return (self.a, self.b, self.mod) == (autre.a, autre.b, autre.p)
177
178     def __call__(self, x):
179         """
180         Retourne y pour un x donné, modulo p.
181
182         Arguments:
183             x : int
184                 La coordonnée x du point.
185
186         Retourne:
187             int : La coordonnée y du point sur la courbe elliptique.
188
189         Lève une exception ValueError si aucune solution n'existe pour ce x.
190         """
191
192         y2 = (x**3 + self.a * x + self.b) % self.mod
193         try:
194             y_pos = residu_quadratique(y2, self.mod)
195             return y_pos
196
197         except Exception:
198             raise ValueError(f"Aucune solution pour x = {x} dans Z/{self.mod}Z")
199
```

```
200
201     def appartient(self, P):
202         """
203         Vérifie si un point P appartient à la courbe elliptique.
204
205         Arguments:
206             P : Point
207                 Le point à vérifier.
208
209         Retourne:
210             bool : True si le point appartient à la courbe, False sinon.
211         """
212
213         if P.isInf():  # Le point à l'infini appartient toujours à la courbe
214            return True
215
216         x = P.x
217         y = P.y
218         return y == self(x) or y == self.mod - self(x)
```

```
    def point_aleatoire(self, inf=False, sup=False):
        """
        Génère un point aléatoire sur la courbe elliptique.

        Arguments:
            inf : int, optionnel
                La limite inférieure de l'intervalle pour la coordonnée x.
            sup : int, optionnel
                La limite supérieure de l'intervalle pour la coordonnée x.

        Retourne:
            Point : Un point aléatoire sur la courbe elliptique.
        """

        if not inf:
            inf = -self.mod
        if not sup:
            sup = self.mod

        i = random.randint(0, 1) # Au hasard entre positif et négatif

        boucle_max = self.mod
        boucle = 0

        while boucle < boucle_max:
            x = random.randint(inf, sup)
            try:
                y_pos = self(x)  # Utiliser la méthode __call__ mise à jour
                y_neg = self.mod - y_pos
                return Point(x, y_pos, self.mod) if i else Point(x, y_neg, self.mod)
            except ValueError:
                pass  # Si n'est pas un résidu quadratique, on essaie un autre x
            boucle += 1
        print("erreur")
```

```
255
256     def plus(self, P, Q):
257         """
258         Effectue l'addition de deux points P et Q sur la courbe elliptique.
259
260         Arguments:
261             P : Point
262                 Le premier point à additionner.
263             Q : Point
264                 Le second point à additionner.
265
266         Retourne:
267             Point : Le résultat de l'addition des deux points.
268
269         Lève une exception si l'un des points ne appartient pas à la courbe.
270         """
271
272         assert self.appartient(P)
273         assert self.appartient(Q)
274
275         if P.isInf():
276             return Q.copy()
277
278         if Q.isInf():
279             return P.copy()
280
281         if P == Q.opose():
282             return Point(None, None)
283
284         if P != Q:
285             a_plus = (Q.y - P.y) % self.mod
286             b_plus = (Q.x - P.x) % self.mod
287             l = (a_plus*(inv_mod(b_plus, self.mod))) % self.mod
288
289         else:  # Si P = Q
290             a_plus = (3*(P.x**2) + self.a) % self.mod
291             b_plus = (2*P.y) % self.mod
292             l = (a_plus*inv_mod(b_plus, self.mod)) % self.mod
293
294         x3 = (l**2 - P.x - Q.x) % self.mod
295
296         y3 = (l*(P.x - x3) - P.y) % self.mod
297
298         R = Point(x3, y3, self.mod)
299         assert self.appartient(R)
300         return R
301
```

```
302     def scalaire(self, n, P):
303         """
304         Calcule n * P (multiplication d'un point par un scalaire) en utilisant l'exponentiation rapide.
305
306         Arguments:
307             n : int
308                 Le scalaire par lequel multiplier le point.
309             P : Point
310                 Le point à multiplier.
311
312         Retourne:
313             Point : Le résultat de la multiplication scalaire.
314
315         Lève une exception si le point n'appartient pas à la courbe ou si n n'est pas un entier.
316         """
317
318         assert self.appartient(P)  # Vérification si point appartient à la courbe
319         assert type(n) == int # Ne marche que si n est entier
320
321         if n < 0:
322             return self.mul(-n, -P)  # On prend l'opposé du point
323
324         # Initialisation
325         Q = P.copy()
326         R = Point(None, None, self.mod)
327
328         while n > 0:
329             if n % 2 == 1: # Si n est impaire
330                 R = self.plus(R, Q) # R = R+Q
331             Q = self.plus(Q, Q) # Q = 2*Q
332             n //= 2 # On divise n par deux
333
334         return Point(R.x, R.y, self.mod)
335
```

```python
    def afficher_courbe(self, xmin=False, xmax=False):
        """
        Trace la courbe elliptique dans l'intervalle [xmin, xmax] modulo p.

        Arguments:
            xmin : int, optionnel
                La borne inférieure de l'intervalle pour x. Par défaut, -10.
            xmax : int, optionnel
                La borne supérieure de l'intervalle pour x. Par défaut, 10.

        Retourne:
            None : Affiche un graphique avec les points de la courbe elliptique.
        """

        if not xmin:
            xmin = -self.mod
        if not xmax:
            xmax = self.mod

        if self.mod > 1000:
            x_vals = [i for i in range(1000)]
        else:
            x_vals = np.arange(self.mod)

        points = []

        for x in x_vals:
            try:
                y_pos = self(x)  # Utiliser la méthode __call__ mise à jour
                points.append((x, y_pos))
                points.append((x, self.mod - y_pos))  # -y est aussi une solution
            except ValueError:
                pass  # Si n'est pas un résidu quadratique, on passe au x suivant


        # Tracer les points sur le graphique
        plt.figure(figsize=(8, 6))
        for (x, y) in points:
            plt.plot(x, y, 'bo')  # Point positif
            plt.plot(x, -y % self.mod, 'ro')  # Point négatif modulo p

        titre = f"Courbe elliptique : y^2 = x^3 + {self.a}x + {self.b} modulo {self.mod}"
        if len(titre) > 100:
            plt.title("Courbe elliptique modulo")
        else:

            plt.title(titre)
```

```python
class Point:
    """
   Représente un point sur une courbe elliptique avec des coordonnées (x, y).
    """


    def __init__(self, x, y, p=mod_zero):
        """
        Initialise un point avec les coordonnées (x, y) et le modulo p.

        Arguments:
           x : int
               La coordonnée x du point.
           y : int
               La coordonnée y du point.
           p : int, optionnel
               Le modulo pour la courbe elliptique.
        """

        self.x = x % p if x != None else x
        self.y = y % p if y != None else y
        self.p = p

    def isInf(self): # Pour faire la différence entre le point infini et les autres
        """
        Vérifie si le point est le point à l'infini.

        Retourne:
           bool : True si le point est à l'infini, False sinon.
       """

        return (self.x, self.y) == (None, None)

    def __str__(self):
        """
        Retourne une chaîne de caractères représentant le point.

        Retourne:
           str : La représentation du point sous forme de chaîne.
       """

        return "0" if self.isInf() else f"({self.x}, {self.y})"
```

```
428     def __repr__(self):
429         """
430         Retourne une représentation plus détaillée du point.
431
432         Retourne:
433             str : La représentation détaillée du point.
434         """
435
436         return str(self)
437
438     def __eq__(self, Q):
439         """
440        Vérifie si deux points sont égaux.
441
442        Arguments:
443            Q : Point
444                Le point à comparer avec l'objet courant.
445
446        Retourne:
447            bool : True si les points sont égaux, False sinon.
448        """
449
450         return (self.p, self.x, self.y) == (Q.p, Q.x, Q.y)
451
452     def opose(self):
453         """
454         Retourne l'opposé d'un point sur la courbe elliptique.
455
456         Retourne:
457            Point : L'opposé du point courant.
458         """
459
460        if self.isInf():
461            return self
462
463        return Point(self.x, (-self.y) % self.p, self.p)
464
465     def copy(self):
466         """
467         Crée une copie du point courant.
468
469         Retourne:
            Point : Une copie du point courant.
         """

         return Point(self.x, self.y, self.p)
```

```
#Fichier Chiffrement.py
# -*- coding: utf-8 -*-
"""
Created on Sat Mar 16 19:16:24 2025

@author: maxime
"""

import random
import hashlib
import ECC
from Crypto.Cipher import AES
from Crypto.Util.Padding import pad, unpad
import base64

class DH_AES:
    """
    Classe gérant l'échange de clé Diffie-Hellman et le chiffrement AES avec Ksym.
    """

    def __init__(self, a, b, mod):
        """
        Initialise une courbe elliptique avec les paramètres donnés et génère un point de base.

        Arguments:
            a : int
                Le paramètre 'a' de la courbe elliptique.
            b : int
                Le paramètre 'b' de la courbe elliptique.
            mod : int
                Le modulaire de la courbe elliptique.
        """

        self.CE = ECC.CourbeElliptique(a, b, mod)
        self.G = self.CE.point_aleatoire()
        assert self.CE.appartient(self.G)

```

```python
    def generation_cles(self):
        """
        Génère une paire de clés publiques et privées pour Alice et Bob.

        Retourne:
            tuple : (pkA, skA, pkB, skB)
                - pkA : Point, clé publique d'Alice.
                - skA : int, clé secrète d'Alice.
                - pkB : Point, clé publique de Bob.
                - skB : int, clé secrète de Bob.
        """

        skA = random.randint(1, self.CE.mod)
        pkA = self.CE.scalaire(skA, self.G)
        skB = random.randint(1, self.CE.mod)
        pkB = self.CE.scalaire(skB, self.G)
        return pkA, skA, pkB, skB

    def derive_cle_secrete(self, Pk, Sk):
        """
        Calcule une clé secrète partagée en utilisant Diffie-Hellman.
        Retourne la clé secrète partagée dérivée, hachée avec SHA-256 pour garantir une longueur fixe de 256 bits.

        Arguments:
            Pk : Point
                La clé publique de l'autre partie.
            Sk : int
                La clé secrète de l'utilisateur.

        Retourne:
            bytes : La clé secrète partagée dérivée et hachée.
        """

        K = self.CE.scalaire(Sk, Pk)
        K_bytes = str(K.x).encode()
        return hashlib.sha256(K_bytes).digest()
```

```
74      def chiffrement_message(self, message, ksym):
75          """
76          Chiffre un message avec AES en utilisant la clé secrète partagée Ksym.
77
78          Arguments:
79              message : str
80                  Le message à chiffrer.
81              ksym : bytes
82                  La clé secrète partagée utilisée pour le chiffrement.
83
84          Retourne:
85              tuple : (ciphertext, iv)
86                  - ciphertext : str, le message chiffré encodé en base64.
87                  - iv : str, le vecteur d'initialisation (IV) utilisé pour le chiffrement, encodé en base64.
88          """
89
90          ksym = ksym[:32]
91          cipher = AES.new(ksym, AES.MODE_CBC)
92          message_padded = pad(message.encode(), AES.block_size)
93          ciphertext = cipher.encrypt(message_padded)
94          return base64.b64encode(ciphertext).decode(), base64.b64encode(cipher.iv).decode()
95
96      def dechiffrement_message(self, ciphertext, iv, ksym):
97          """
98          Déchiffre un message avec AES en utilisant la clé secrète Ksym.
99
100         Arguments:
101             ciphertext : str
102                 Le message chiffré encodé en base64.
103             iv : str
104                 Le vecteur d'initialisation (IV) encodé en base64.
105             ksym : bytes
106                 La clé secrète partagée utilisée pour le déchiffrement.
107
108         Retourne:
109             str : Le message déchiffré.
110         """
111
112         ksym = ksym[:32]
113         cipher = AES.new(ksym, AES.MODE_CBC, iv=base64.b64decode(iv))
114         decrypted_data = unpad(cipher.decrypt(base64.b64decode(ciphertext)), AES.block_size)
115         return decrypted_data.decode()
116
```

```python
class EC_ElGamal:
    """
    Classe gérant le chiffrement et déchiffrement basé sur les courbes elliptiques (EC-ElGamal).
    """

    def __init__(self, a, b, mod):
        """
        Initialise une courbe elliptique avec les paramètres donnés et génère un point de base.

        Arguments:
            a : int
                Le paramètre 'a' de la courbe elliptique.
            b : int
                Le paramètre 'b' de la courbe elliptique.
            mod : int
                Le modulaire de la courbe elliptique.
        """

        self.CE = ECC.CourbeElliptique(a, b, mod)
        self.G = self.CE.point_aleatoire()
        assert self.CE.appartient(self.G)

    def generation_cles(self):
        """
        Génère une paire de clés publiques et privées pour Alice et Bob.

        Retourne:
            tuple : (pkA, skA, pkB, skB)
                - pkA : Point, clé publique d'Alice.
                - skA : int, clé secrète d'Alice.

                - pkB : Point, clé publique de Bob.
                - skB : int, clé secrète de Bob.
        """

        skA = random.randint(1, self.CE.mod)
        pkA = self.CE.scalaire(skA, self.G)
        skB = random.randint(1, self.CE.mod)

        pkB = self.CE.scalaire(skB, self.G)
        return pkA, skA, pkB, skB
```

```python
    def message_to_point(self, message):
        """
        Convertit un message en un point sur la courbe elliptique en utilisant la méthode de rejet.

        Arguments:
            message : str
                Le message à convertir en point sur la courbe elliptique.

        Retourne:
            tuple : (point, decalage)
                - point : ECC.Point, le point sur la courbe elliptique représentant le message.
                - decalage : int, le décalage appliqué au message pour obtenir un point valide.

        Lève:
            ValueError : Si le message ne peut pas être converti en point après plusieurs tentatives.
        """

        x_original = int.from_bytes(message.encode(), 'big') % self.CE.mod
        x = x_original
        decalage = 0
        boucle_max = 1000
        boucle = 0

        while boucle < boucle_max:
            try:
                y = self.CE(x)
                return ECC.Point(x, y, self.CE.mod), decalage
            except ValueError:
                x = (x + 1) % self.CE.mod
                decalage += 1
                boucle += 1

        raise ValueError("Le message ne peut être converti")
```

```
156         return pkA, skA, pkB, skB
```

```python
    def point_to_message(self, point, decalage=0):
        """
        Convertit un point elliptique en message texte original en retirant le décalage appliqué.

        Arguments:
            point : ECC.Point
                Le point sur la courbe elliptique à convertir en message.
            decalage : int
                Le décalage appliqué lors de la conversion du message en point.

        Retourne:
            str : Le message récupéré depuis le point elliptique, ou un message d'erreur si la conversion échoue.
        """

        x_original = (point.x - decalage) % self.CE.mod
        try:
            message = x_original.to_bytes((x_original.bit_length() + 7) // 8, 'big').decode()
            return message
        except UnicodeDecodeError:
            return f"[Message illisible], {x_original}"

    def chiffrement_message(self, pkA, skB, message):
        """
        Chiffre un message en le transformant en point elliptique et en utilisant Diffie-Hellman.

        Arguments:
            pkA : ECC.Point
                La clé publique de la partie à qui le message est destiné.
            skB : int
                La clé privée de l'envoyeur
            message : str
                Le message à chiffrer.

        Retourne:
            tuple : (M_chiffre, decalage)
                - M_chiffre : ECC.Point, le message chiffré
                - decalage : int, le décalage appliqué lors de la conversion du message.
        """

        M, decalage = self.message_to_point(message)
        skB_pkA = self.CE.scalaire(skB, pkA)
        M_chiffre = self.CE.plus(skB_pkA, M)
        return M_chiffre, decalage
```

```
237     def dechiffrement_message(self, skA, pkB, M_chiffre, decalage=0):
238         """
239         Déchiffre un message chiffré sous forme de point elliptique.
240 
241         Arguments:
242             skA : int
243                 La clé secrète de l'utilisateur (utilisée pour déchiffrer).
244             pkB : ECC.Point
245                 La clé publique de l'envoyeur
246             M_chiffre : ECC.Point
247                 Le message chiffré sous forme de point elliptique
248             decalage : int
249                 Le décalage appliqué lors de la conversion du message en point.
250 
251         Retourne:
252             str : Le message déchiffré.
253         """
254 
255         skA_pkB = self.CE.scalaire(skA, pkB)
256         skA_pkB_inv = skA_pkB.opose()
257         M_dechiffre = self.CE.plus(M_chiffre, skA_pkB_inv)
            return self.point_to_message(M_dechiffre, decalage)
```

```python
#Fichier main.py
# -*- coding: utf-8 -*-
"""
Created on Sun Mar 16 15:20:19 2025

@author: maxime
"""

import Chiffrement

def simuler_chiffrement_DH_AES(message):
    """
    Simule l'échange de clé Diffie-Hellman et chiffre/déchiffre un message avec AES.

    - Alice et Bob génèrent leurs clés publiques et privées.
    - Ils échangent leurs clés publiques et dérivent une clé secrète commune (Ksym).
    - Alice chiffre le message avec AES en utilisant Ksym et l'envoie à Bob.
    - Bob déchiffre le message avec la même clé Ksym.

    Arguments:
        message (str): Le message à chiffrer.

    Lève:
        ValueError: Si l'échange de clé échoue ou si le message déchiffré ne correspond pas à l'original.
    """

    # Paramètres de la courbe elliptique
    a = 109454571331697278617670725030735128145969349647868738157201323556196022393856
    b = 107744541122042688792155207242782455150382764043089114141096634497567301547839
    mod = 109454571331697278617670725030735128145969349647868738157201323556196022393859
    chiffrage = Chiffrement.DH_AES(a, b, mod)

    # Génération des clés d'Alice et Bob
    pkA, skA, pkB, skB = chiffrage.generation_cles()

    # Échange de clés et dérivation de Ksym
    print(f"Bob envoi à Alice : {pkB}")
    print(f"Alice envoi à Bob : {pkA}")
    ksym_Alice = chiffrage.derive_cle_secrete(pkB, skA)
    print(f"Alice calcul Ksym : {ksym_Alice}")
    ksym_Bob = chiffrage.derive_cle_secrete(pkA, skB)
    print(f"Bob calcul Ksym : {ksym_Bob}")

    if ksym_Alice == ksym_Bob:
        print("L'obtention de Ksym a réussi")
        ksym = ksym_Bob
    else:
        raise ValueError("Erreur d'échange de clés.")

```

```
    # Chiffrement et transmission du message
    ciphertext, iv = chiffrage.chiffrement_message(message, ksym)
    print("Alice envoie le message chiffré à Bob")
    print(f"Message chiffré : {ciphertext}")

    # Déchiffrement et vérification
    message_decrypte = chiffrage.dechiffrement_message(ciphertext, iv, ksym)
    print("Bob décrypte le message")
    print(f"Message déchiffré : {message_decrypte}")

    if message_decrypte == message:
        print("Le chiffrement et déchiffrement ont réussi !")
    else:
        raise ValueError("Erreur : Le message déchiffré ne correspond pas au message original.")


def simuler_chiffrement_EC_ElGamal(message):
    """
    Simule le chiffrement et déchiffrement d'un message en utilisant EC-ElGamal.

    - Alice génère une clé publique et l'envoie à Bob.
    - Bob chiffre le message en utilisant la clé publique d'Alice et envoie son propre pkB à Alice.
    - Alice utilise sa clé privée pour déchiffrer le message.

    Arguments:
        message (str): Le message à chiffrer.

    Retourne:
        str: Le message déchiffré.

    Lève:
        ValueError: Si le chiffrement ou déchiffrement échoue.
    """

    # Paramètres de la courbe elliptique
    a = 109454571331697278617670725030735128145969349647868738157201323556196022393856
    b = 107744541122042688792155207242782455150382764043089114141096634497567301547839
    mod = 109454571331697278617670725030735128145969349647868738157201323556196022393859
    chiffrage = Chiffrement.EC_ElGamal(a, b, mod)
```

```
90
91      # Génération des clés d'Alice et Bob
92      pkA, skA, pkB, skB = chiffrage.generation_cles()
93
94      # Chiffrement du message par Bob
95      try:
96          print(f"Alice envoie sa clé publique à Bob : {pkA}")
97          print("Bob chiffre le message grâce à sa clé privée et à la clé publique d'Alice")
98          (M_chiffre, decalage) = chiffrage.chiffrement_message(pkA, skB, message)
99          print(f"Bob envoie à Alice sa clé publique : {pkB} ; et le message chiffré : {M_chiffre} avec le décalage : {decalage}")
100     except ValueError as e:
101         return f"Erreur de chiffrement : {e}"
102
103     # Déchiffrement du message par Alice
104     try:
105         print("Alice déchiffre le message en utilisant sa clé privée, la clé publique de Bob, le message chiffré et le décalage")
106         M_dechiffre = chiffrage.dechiffrement_message(skA, pkB, M_chiffre, decalage)
107         return M_dechiffre
108     except ValueError as e:
109         return f"Erreur de déchiffrement : {e}"
110
111 # Exemple
112 message1 = "Je suis heureux que cela fonctionne, vive mon TIPE !"
113 message2 = "Je suis heureux."
114 #simuler_chiffrement_DH_AES(message1)
115 simuler_chiffrement_EC_ElGamal(message2)
```